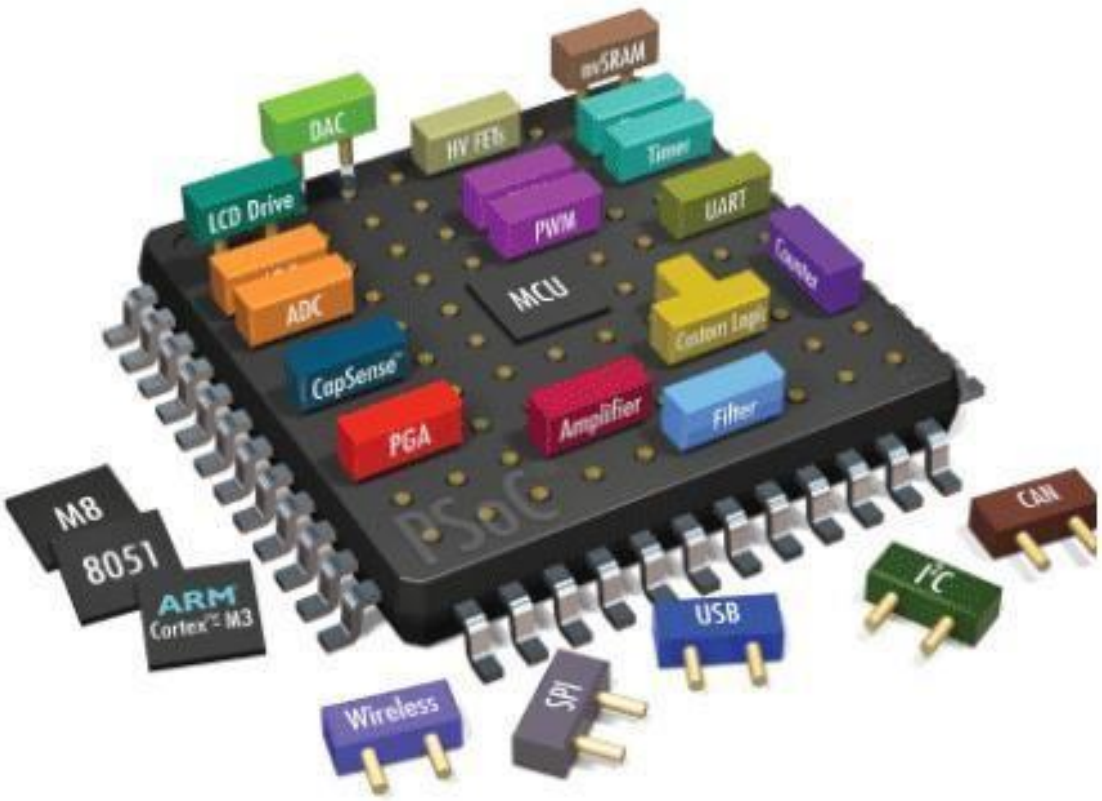
HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Embedded Systems

Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.



Dated:

Week 06

Semester:

Fall 2024

Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Introduction to Keil:

Embedded system means some combination of computer hardware and programmable software which is specially designed for a particular task like displaying message on LCD. It involves hardware (8051 microcontroller) and software (the code written in assembly language).

Some real-life examples of embedded systems may involve ticketing machines, vending machines, temperature controlling unit in air conditioners etc. Microcontrollers are nothing without a Program in it.

One of the important parts in making an embedded system is loading the software/program we develop into the microcontroller. Usually it is called “burning software” into the controller. Before “burning a program” into a controller, we must do certain prerequisite operations with the program. This includes writing the program in assembly language or C language in a text editor like notepad, compiling the program in a compiler and finally generating the hex code from the compiled program. Earlier people used different software’s /applications for all these 3 tasks. Writing was done in a text editor like notepad/ WordPad, compiling was done using a separate software (probably a dedicated compiler for a particular controller like 8051), converting the assembly code to hex code was done using another software etc. It takes lot of time and work to do all these separately, especially when the task involves lots of error debugging and reworking on the source code.

The µVision IDE is the easiest way for most developers to create embedded applications using the Keil development tools. The new Keil µVision4 IDE has been designed to enhance developer's productivity, enabling faster, more efficient program development.

Keil Micro Vision is a free software which solves many of the main points for an embedded program developer. This software is an integrated development environment (IDE), which integrated a text editor to write programs, a compiler and it will convert your source code to hex files too. µVision4 introduces a flexible window management system, enabling us to drag and drop individual windows anywhere on the visual surface including support for Multiple Monitors.

Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Embedded Systems Vs General Computing Systems

General Purpose System	Embedded System
A system which is a combination of generic hardware and General Purpose Operating System for executing a variety of applications	A system which is a combination of special purpose hardware and embedded OS for executing a specific set of applications
Contain a General Purpose Operating System (GPOS)	May or may not contain an operating system for functioning
Applications are alterable (programmable) by user (It is possible for the end user to re-install the Operating System, and add or remove user applications)	The firmware of the embedded system is pre-programmed and it is non-alterable by end-user (There may be exceptions for systems supporting OS kernel image flashing through special hardware settings)
Performance is the key deciding factor on the selection of the system. Always 'Faster is Better'	Application specific requirements (like performance, power requirements, memory usage etc) are the key deciding factors
Less/not at all tailored towards reduced operating power requirements, options for different levels of power management.	Highly tailored to take advantage of the power saving modes supported by hardware and Operating System
Response requirements are not time critical	For certain category of embedded systems like mission critical systems, the response time requirement is highly critical
Need not be deterministic in execution behavior	Execution behavior is deterministic for certain type of embedded systems like 'Hard Real Time' systems

source from internet

C51 Development Tools

Keil development tools for the 8051-microcontroller family support every level of developer from the professional applications engineer to the student just learning about embedded software development. The industry-standard Keil C Compilers, Macro Assemblers, Debuggers, Real-time Kernels, and Single-board Computers support ALL 8051-compatible derivatives and help you get your projects completed on schedule.

The following table shows the Keil C51 Product Line and the Components that are included. You may use this information to find the development tool kit that best fits your needs.

Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Introduction

The C51 development tool chains are designed for the professional software developer, but any level of programmer can use them to get the most out of the 8051-microcontroller architecture.

With the C51 tools, embedded applications can be generated for virtually every 8051 variants. Refer to the µVision Device Database for a list of currently supported microcontrollers.

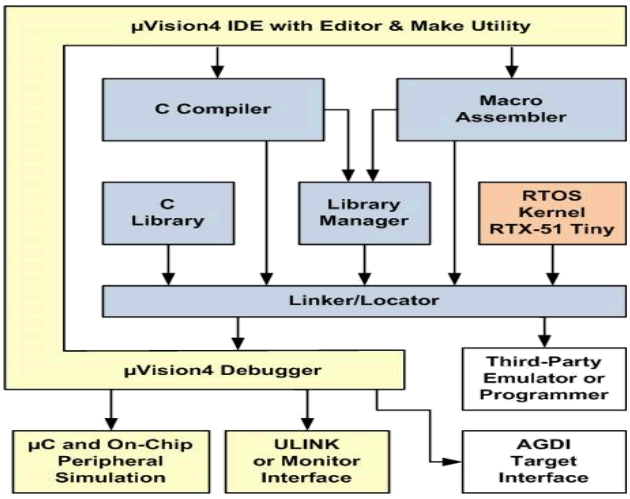
This introduction includes a brief explanation of the:

- Software Development Cycle that describes the steps and tools involved to create a project.
- Development Tools that describe the major features of the Keil C51 development tools including the µVision IDE and Debugger.
- Folder Structure that describes the default location of µVision and the C51 tool chain installation

Development Tools

The Keil C51 development tools offer numerous features and advantages that help you to develop embedded applications quickly and successfully. Find out more about the supported devices and the possible tool combinations available for the different 8051 variants.

The following block diagram shows the components involved in the build process.



µVision Integrated Development Environment (IDE)

The µVision IDE is a window-based software development tool that combines project management and a rich-featured editor with interactive error correction, option setup, make facility, and on-line help. Use µVision to create source files and organize them into a project that defines your target application.

Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

C Compiler

The Keil Cx51 Compiler is a full ANSI implementation of the C programming language and supports all standard features of the C language. In addition, numerous extensions have been included to directly support the 8051 and extended 8051 architectures.

Macro Assembler

The Keil Ax51 Macro Assembler supports the complete instruction set of the 8051 and all 8051 derivatives.

Library Manager

The LIBx51 Library Manager allows you to create the object library from object files created by the compiler and assembler. Libraries are specially formatted, ordered program collections of object modules that may be used by the linker at a later time. When the linker processes a library, only those object modules necessary to create the program are used.

Linker/Locater

The Lx51 Linker/Locater creates the final executable 8051 program and combines the object files created by the compiler or assembler, resolves external and public references, and assigns absolute addresses. In addition, it selects and includes the appropriate run-time library modules.

µVision Debugger

The µVision Debugger is ideally suited for fast and reliable program debugging. The debugger includes a high-speed simulator capable of simulating an entire 8051 system including on-chip peripherals and external hardware.

The µVision Debugger provides several ways to test programs on target hardware:

- Use the Keil ULINK USB-JTAG adapter for downloading and testing your program.
- Install a target monitor on your target system and download your program using the built-in monitor interface of the µVision Debugger.
- Use the Advanced GDI interface to attach and use the µVision Debugger front end with your target system.

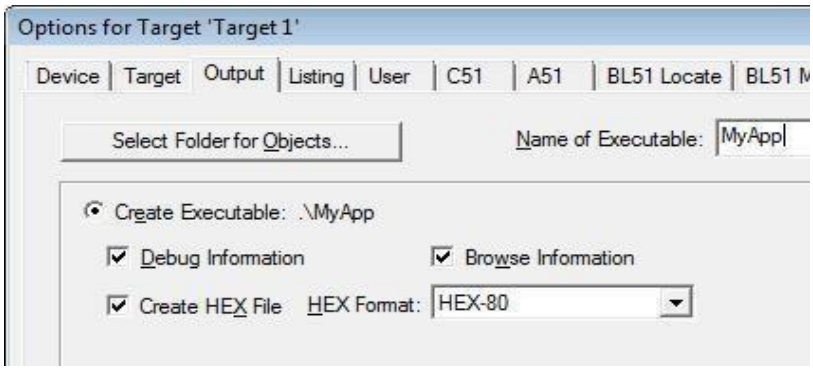
RTOS Kernel

The RTOS Kernel, describes the advantages of using a real-time kernel like the Keil RTX51 Tiny in embedded systems.

Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Creation of HEX File

Some applications require a HEX file to download the application software into the physical device using a Flash programming utility. µVision creates HEX files with each build process when Create HEX File is enabled in the dialog Options for Target Output.



If code banking is used, then the application has to be converted with the OC51 Banked Object File Converter prior to using the OH51 Object/Hex converter.

When the extended LX51 Linker is used, it is mandatory to use the OHx51 Extended Object-HEX Converter to generate an Intel HEX-386 file that contains the common area and all the code banks.

Start Debugging

µVision provides several ways to invoke debugging commands:

- Commands used from the menu Debug or the Debug Toolbar.
- Commands entered manually in the Command Window.
- Commands available from the Context Menu of the Editor or Disassembly window.
- Debug Functions executed from an initialization file.

Start the Debugger

- Use the **Start/Stop Debug Session** button from the **Debug Toolbar** to start or stop a debugging session.
- The current instruction or high-level statement (the one about to execute) is marked with a yellow arrow. For each step-command, the arrow moves to reflect the new current line or instruction.
- Depending on the **Options for Target — Debug** configuration, µVision loads the application program and runs the startup code (**Run to main ()**).
- µVision saves the editor screen layout and restores the screen layout of the last debug

session. When program execution stops, µVision opens an Editor window with the source text or shows MCU instructions in the Disassembly Window.

Execute Commands

- Run the program to the next break point, or type **GO** in the **Command Line**.
    - Halt the program, or press **Esc** while in the **Command Line**
- Click **Reset** from the **Debug Toolbar** or from the **Debug — Reset CPU Menu** or type **RESET** in the **Command Line** to reset the CPU.

Single-Stepping Commands

- To step through the program and into function calls. Alternatively, you can enter **TSTEP** in the **Command Line**, or press **F11**.
- To step over the program and over function calls. Alternatively, you can enter **PSTEP** in the **Command Line**, or press **F10**.
- To step out of the current function. Alternatively, you can enter **OSTEP** in the **Command Line**, or press **Ctrl+F11**.

On-Chip Peripherals

There are a number of techniques you must know to create programs that can use the various on-chip peripherals and features of the 8051 family. Use the code examples provided here to get started working with the 8051.

There is no single standard set of on-chip peripherals for the 8051 family. Instead, 8051 chip vendors use a wide variety of on-chip peripherals to distinguish their parts from each other. The code examples demonstrate how to use the peripherals of a particular chip or family. Be aware that there are more configuration options available than are presented in this text.

Follow the links to the on-chip peripherals:

- **Header Files** - use the include files to define peripheral registers of the device in use.
- **Startup Code** - initializes the microcontroller and transfers control to the **main** function.
- **Special Function Registers** - explains how to use Special Function Registers (SFRs).
- **Register Banks** - explains how to use Register Banks.
- **Interrupt Service Routines** - lists the different interrupt variants on 8051 devices.
- **Interrupt Enable Registers** - shows how to enable the interrupts.
- **Parallel Port I/O** - explains how to use standard I/O ports.
- **Timers/Counters** - explains standard timers and counters.
- **Serial Interface** - explains the implementation of serial UART communication.
- **Watchdog Timer** - use a watchdog timer to recover from hardware or software failures.
- **D/A Converter** - convert a digital output voltage to an analog output value.

- **A/D Converter** - convert an analog input voltage to a digital value.
- **Power Reduction Modes** - put the device into IDLE or POWER DOWN mode.

Startup Code

Startup Code is executed immediately upon RESET of the target system and performs the following operations:

- Depending on the device variant, device specific features are configured.
- Clears data memory (optionally).
- Initializes the reentrant stack and re-entrant stack pointer (optionally).
- Initializes the 8051-hardware stack pointer.
- Transfers control to the variable initialization code or to the main C function.

Differences Between µVision and C

A number of differences exist between ANSI C and the language subset to support features in user- and signal functions.

- µVision does not differentiate between uppercase and lowercase. The names of objects and control statements may be written in either uppercase or lowercase.
- µVision has no preprocessor. Preprocessor directives like #define, #include, and #ifdef are not supported.
- µVision does not support global declarations. Scalar variables must be declared within a function definition. You may define symbols with the DEFINE command and use them like you would use a global variable.
- in µVision, variables may not be initialized when they are declared. Explicit assignment statements must be used to initialize variables.
- µVision functions only support scalar variable types. Structures, arrays, and pointers are not allowed. This applies to the function return type as well as the function parameters.
- µVision functions may only return scalar variable types. Pointers and structures may not be returned.
- µVision functions cannot be called recursively. During function execution, µVision recognizes recursive calls and abort's function execution if one is detected.
- µVision functions may only be invoked directly using the function name. Indirect function calls via pointers are not supported.
- A software program called "Cross-compiler" is used for the conversion of programs written in Embedded C to target processor /controller specific instructions.

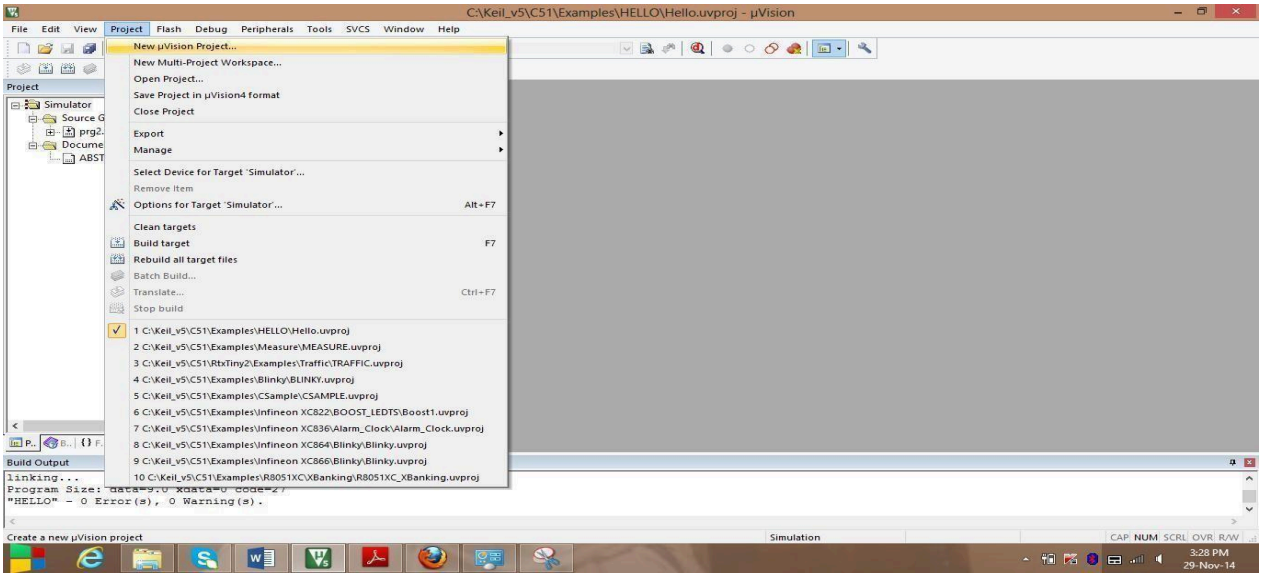
HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

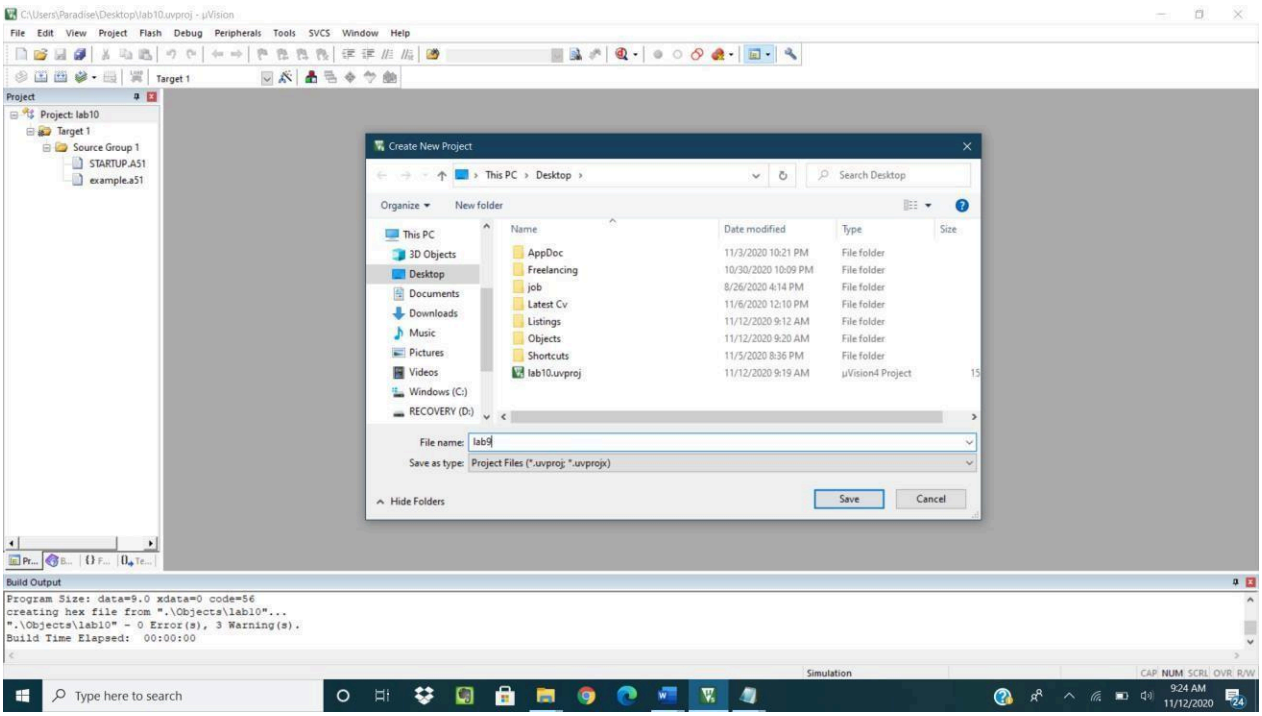
Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Procedure:

Step 1: Create new u vision project

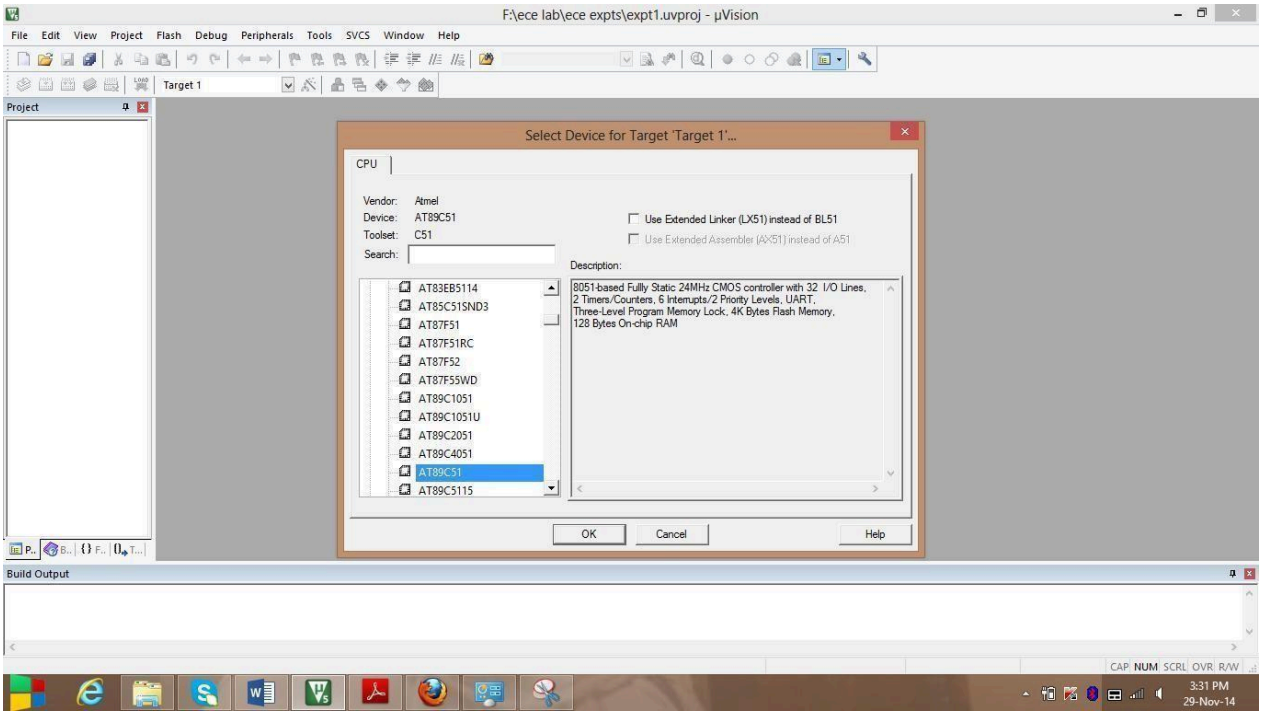


Step 2: Select the existing folder or create lab6 to save the project

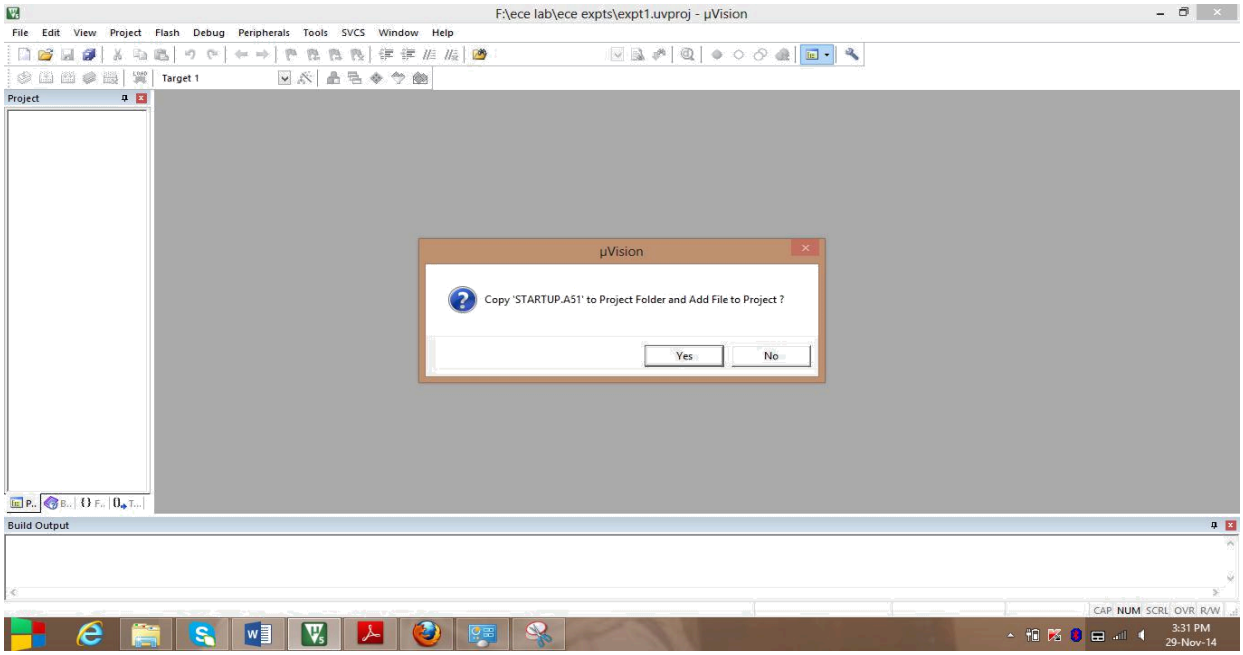


Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Step 3: Select the vendor "NXP" and device "89V51RD2"

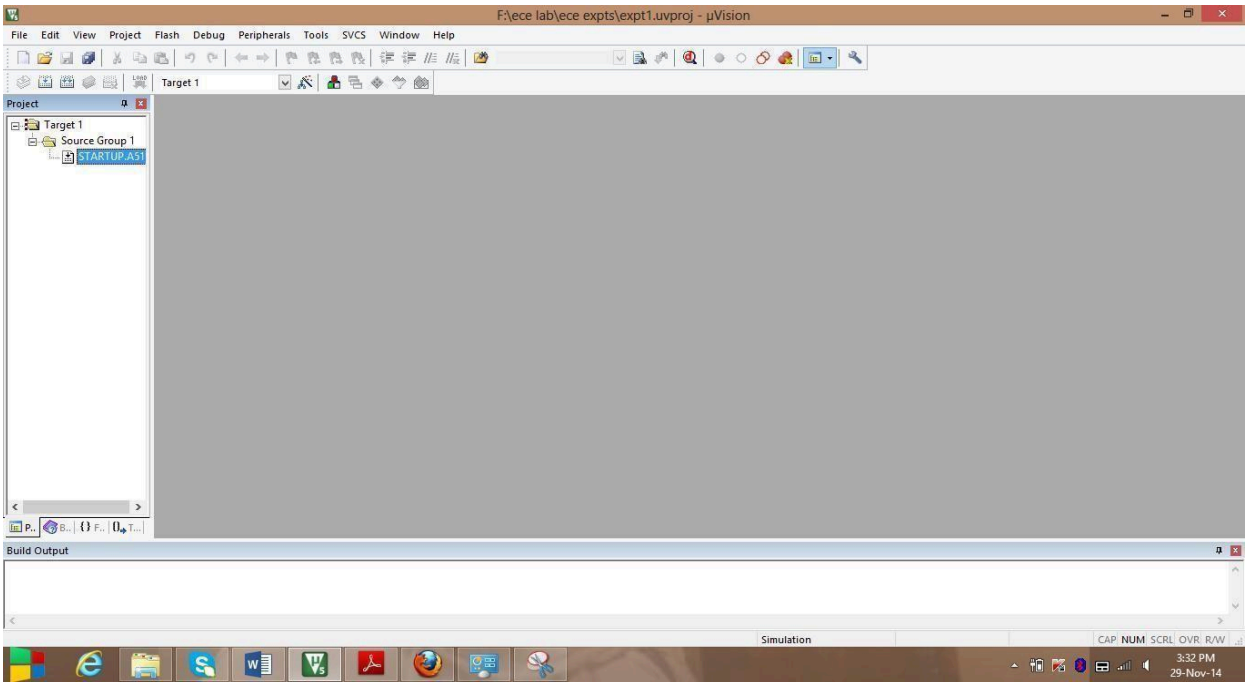


Step 4: Addition of STARTUP.A51 to project folder

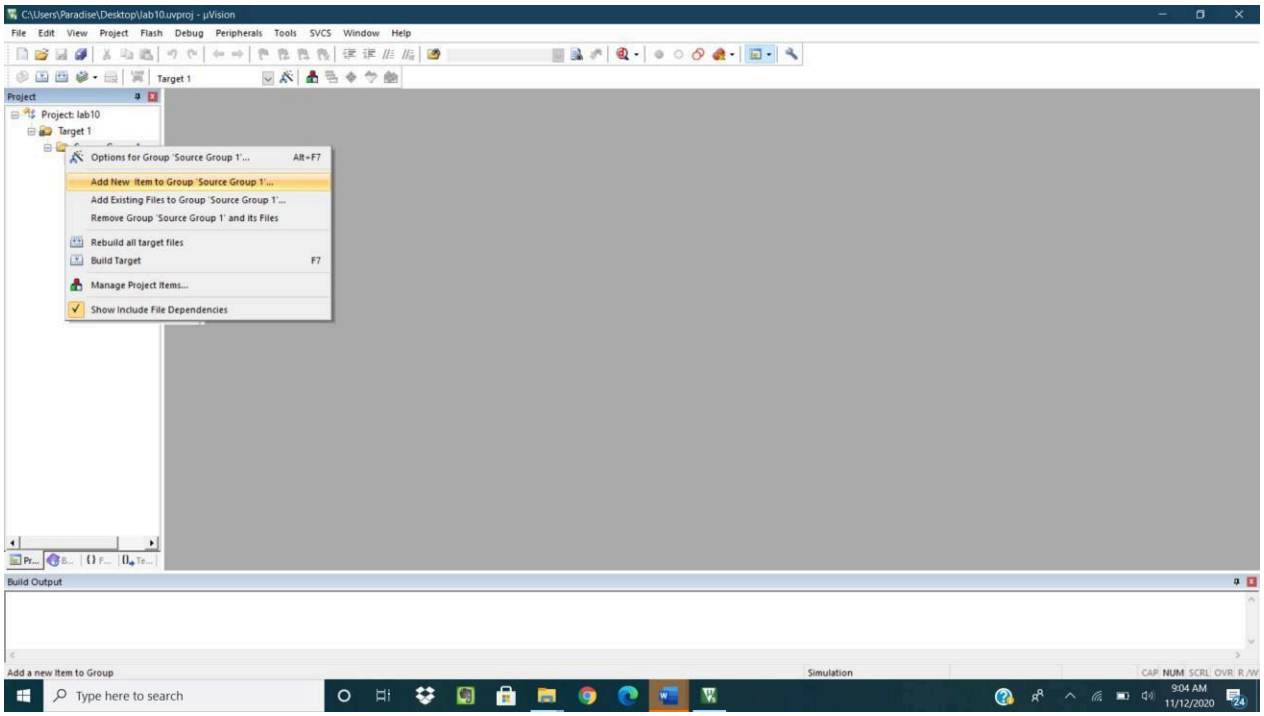


Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Step 5: STARTUP.A51 is added

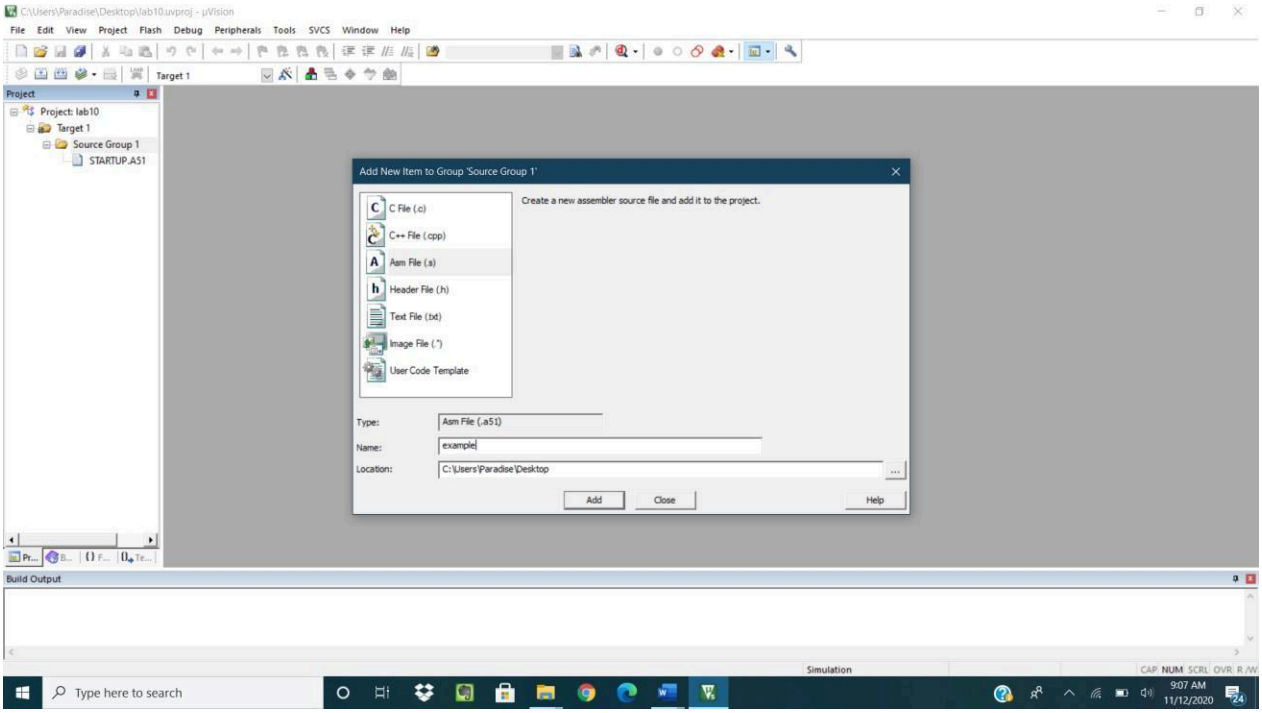


Step 6: Add new file to Source Group1

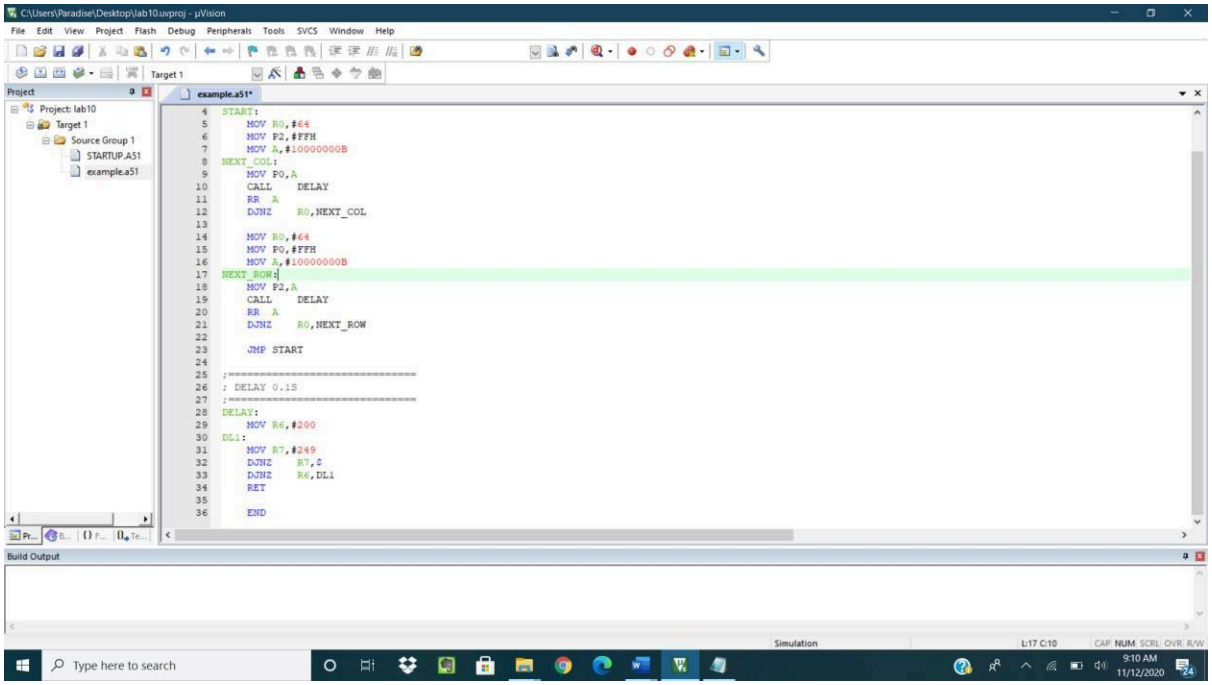


Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Step 7: Write the name of program as example.s now example.s is added to Source Group 1

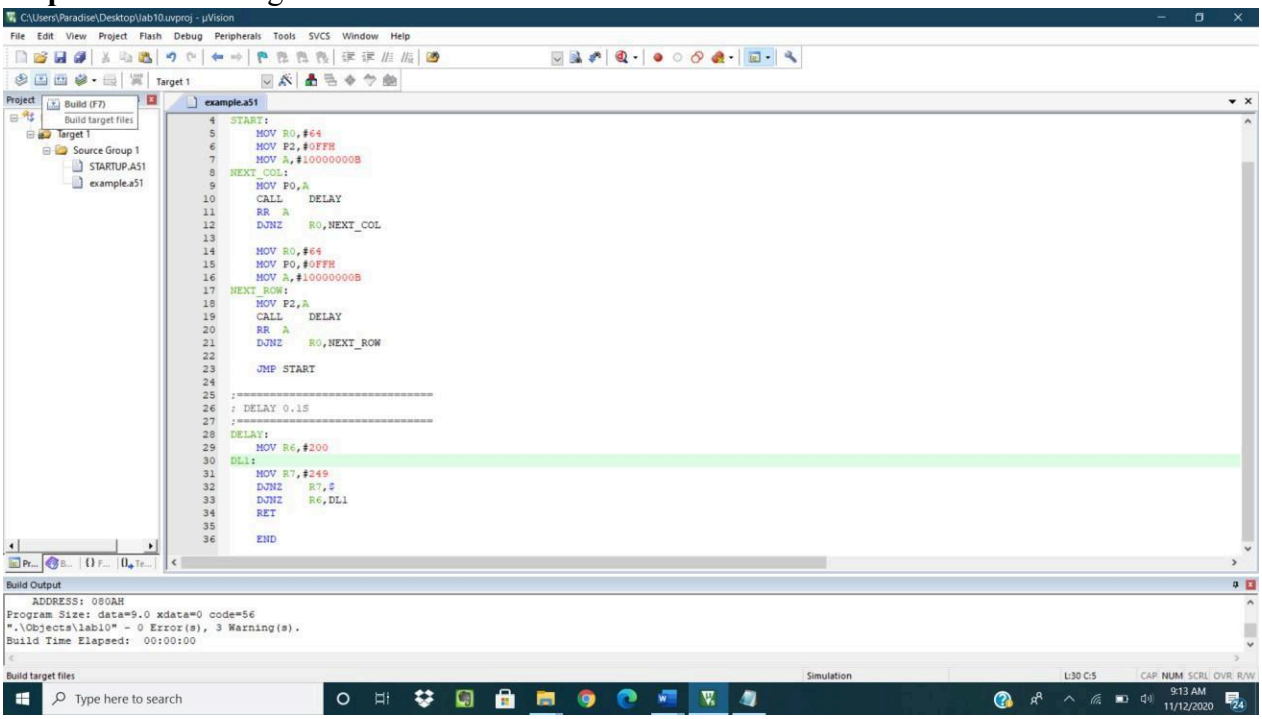


Step 8: Write the Code for the example given in the manual and Save the program.

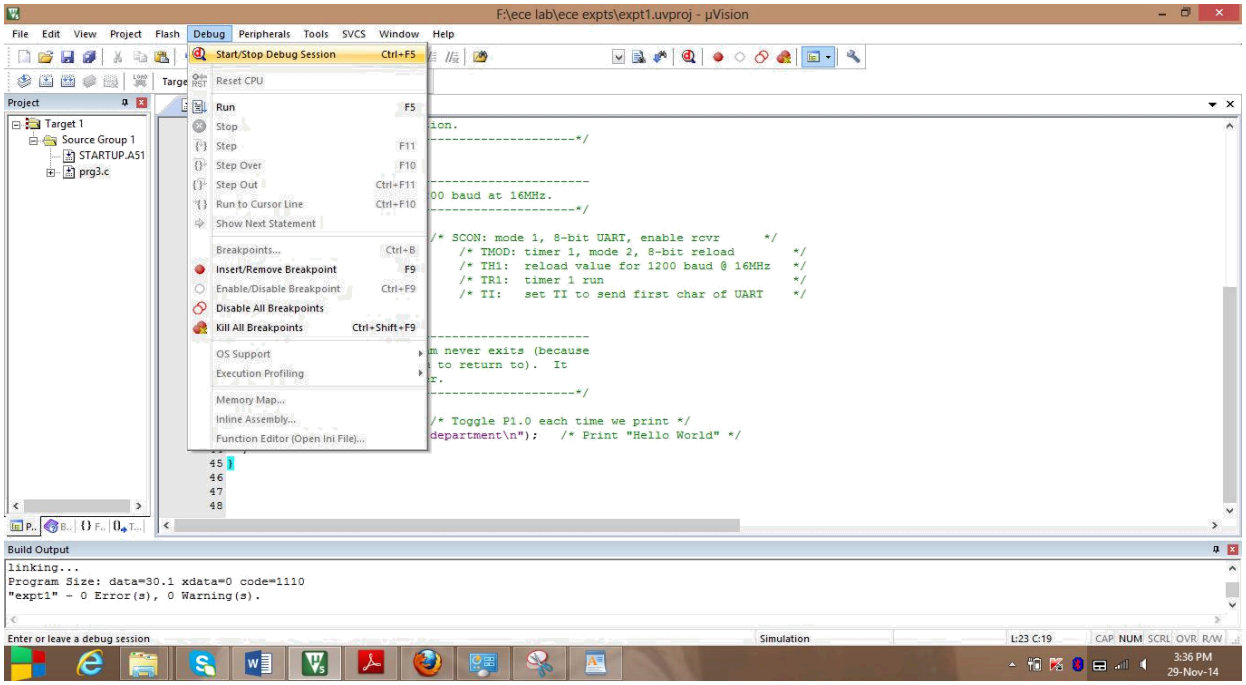


Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Step 9: Build the target.

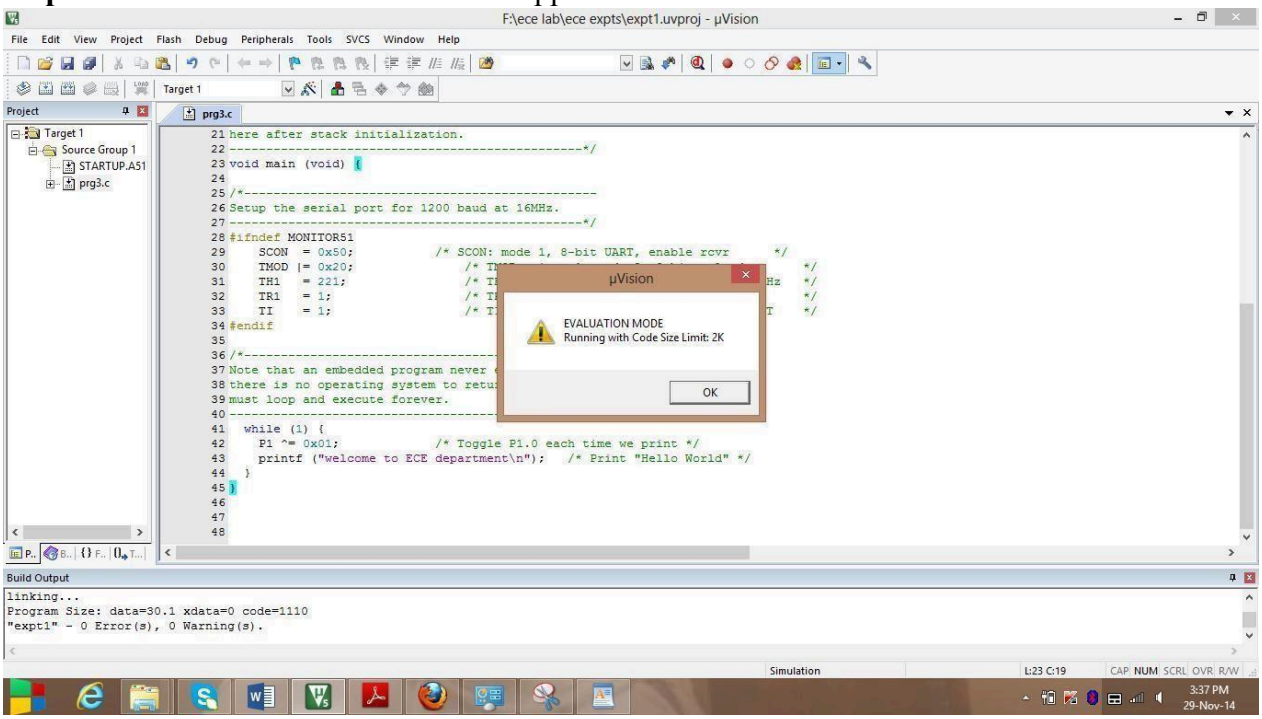


Step 10: Debugging the target.

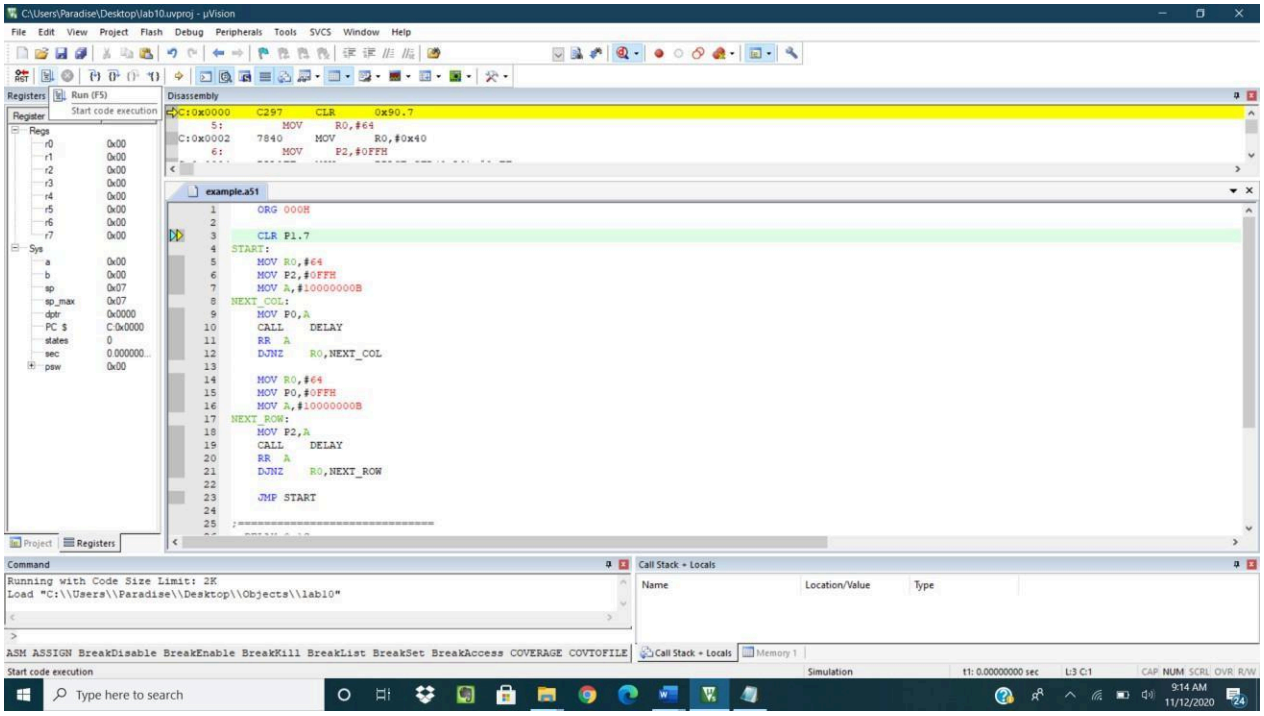


Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Step 11: New window evaluation mode appeared. Press ok

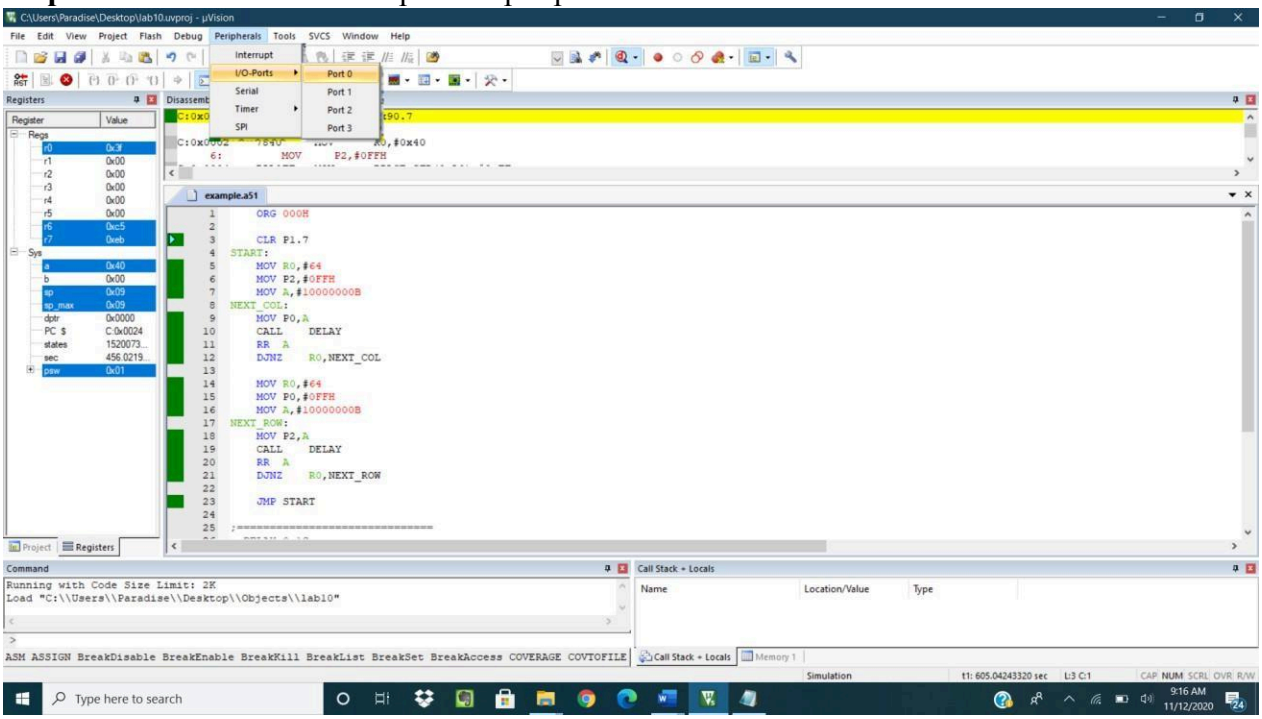


Step 12: Run the program.

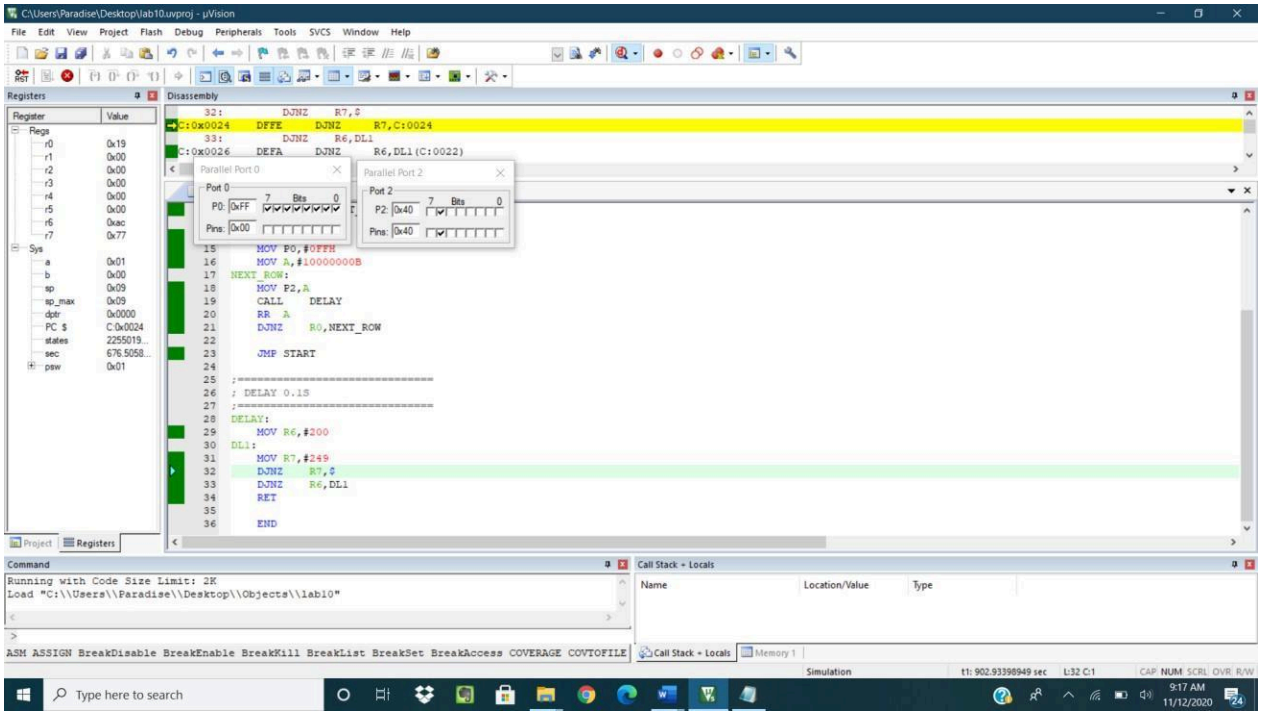


Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Step 13: Select Port2 from i/o ports in peripherals.

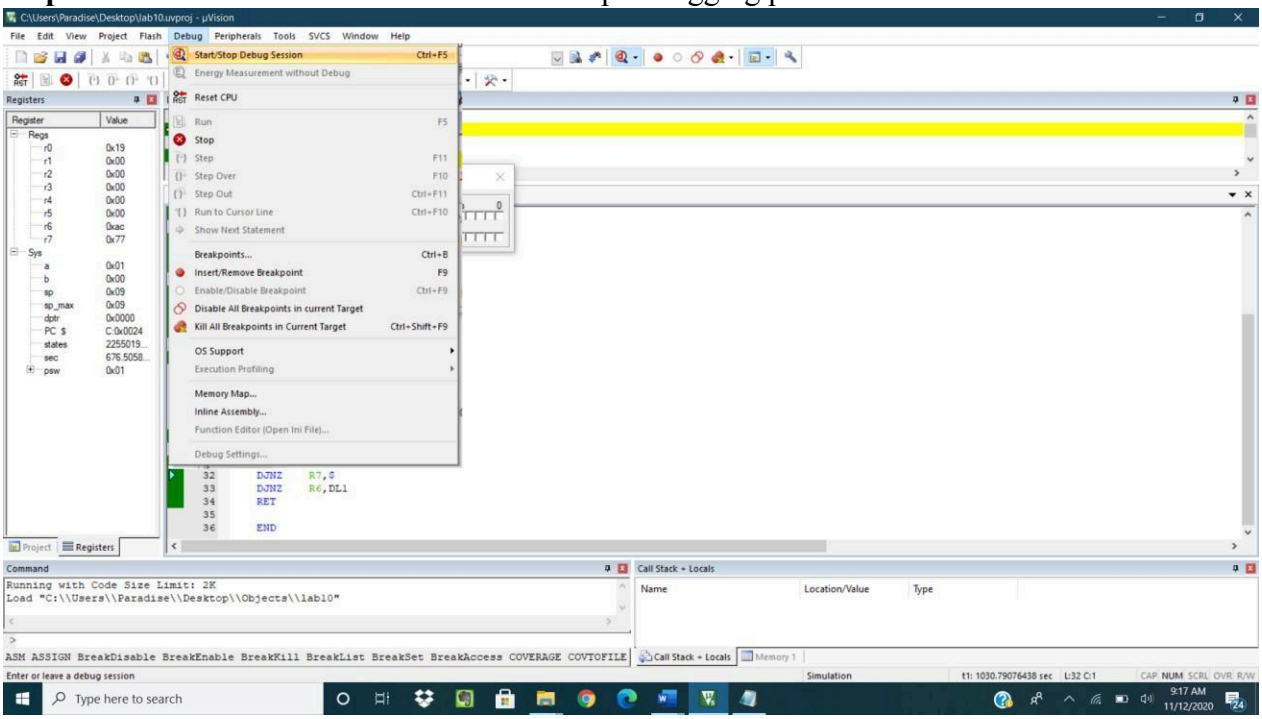


Step 14: Port2 window is displayed. Analyze the output pattern.

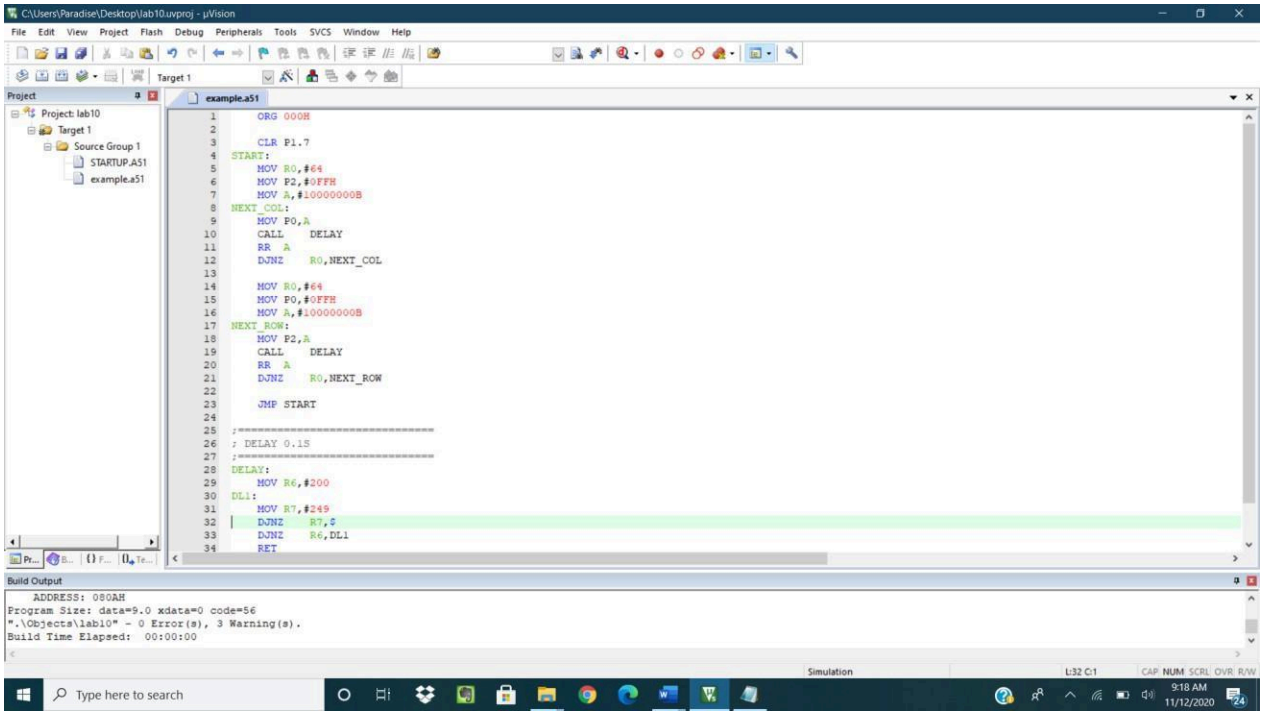


Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Step 15: Great Job. You have done with it. Stop debugging process.



Step 16: After debugging, window appears in this format.



Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

Example:

```
        ORG     000H

        MOV     A,#10000000B
NEXT:
        MOV     P2,A
        CALL    DELAY
        RR      A
        JMP     NEXT

;==============================
; DELAY 0.1S
;==============================
DELAY:
        MOV     R6,#200
DL1:
        MOV     R7,#249
        DJNZ    R7,$
        DJNZ    R6,DL1
        RET

        END
```

Lab Tasks:

1. Write a program which Sequentially and repeatedly turn on/off the 8 LEDs from the leftmost LED to the rightmost LED. The ON time of each LED is 0.1 seconds.
2. Write a program which Sequentially turn on/off the 8 LEDs from the leftmost LED to the rightmost LED. When the rightmost LED is reached, the sequence is inverted. The ON time of each LED is 0.1 seconds.
3. Write a program which Sequentially turn on/off the two LEDs from two ends towards the middle. When the middle reached, the sequence is inverted. The ON time of each LED is 1 seconds.
4. Write a program which toggle between the even led and odd led having the delay of 2 sec.
5. Write a program which Uses the technique of table lookup to show the LED patterns resided in memory. Each pattern stays on 0.1 seconds.
6. Write a program which Turn on LEDs according to the following cases and repeat each case 8 times.
    a. Case 1: Turn on LEDs from left to right.
    b. Case 1: Turn on LEDs from right to left.
    c. Blink two sets (left and right sides) of LEDs alternately.
    d. Blink all of 8 LEDs.

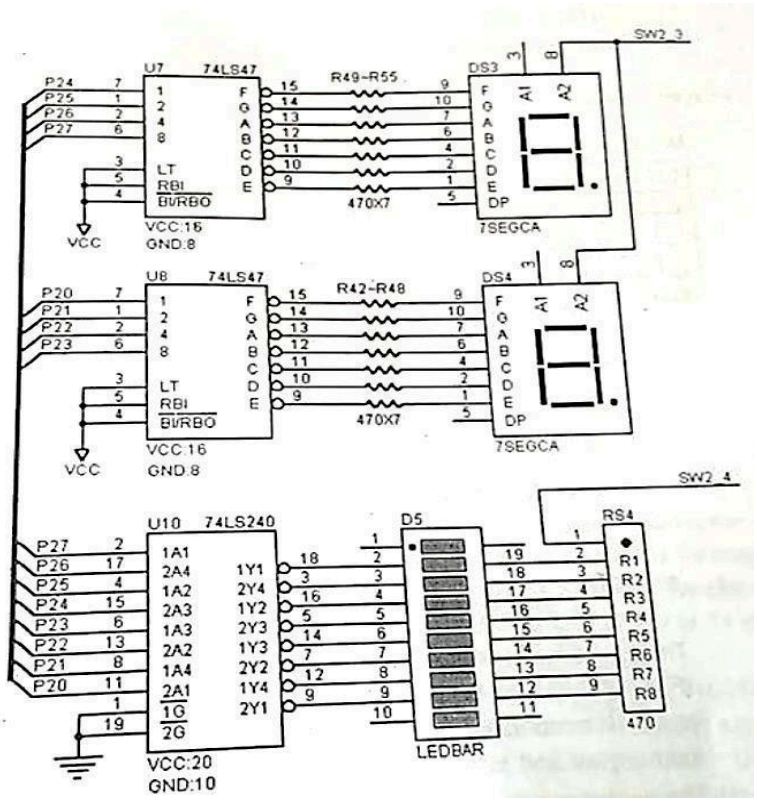
Hint:

HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 06: Introduction to Keil Debugger and simulator with Led Display Control using MTS-51.

1. I/O port 2 controls LED pack D5 on the MTS-51 Trainer. When P2 bit is 1, the corresponding LED is turned ON, When P2 bit is 0, the corresponding LED is turned OFF. Note that the SW-2 switch should be placed in ON position.
2. Circuit diagram



Lab Performance Evaluation:

Read and practice the manual. Performance is evaluated at the end of lab based on successfully running and understanding of examples, tasks and viva using defined rubrics.

Lab Report Evaluation:

Submit the Lab Report by writing all the examples and tasks which must include the following:

1. Brief description about how its work (3-5 lines)
2. The code
3. The results (Screenshot)

Important note: Lab Report must be according to the Lab Report Format available on Google Classroom and must be submitted on time. Two similar reports will get zero marks. So, don't try to copy your fellow reports. Lab report must be submitted in group of three maximum.